From Prediction to Digital Twin: A Propagation-Aware Machine Learning Framework for Flight Delay Modeling

Jonathan Wilson Karan Nindra Irena Austin Val Sokolow

2026-03-13

Abstract
Flight delays remain a persistent challenge in modern aviation, driven by complex interactions between weather, operations, and network dynamics. This study presents a propagation-aware machine learning framework that models delays as a function of upstream aircraft state rather than isolated flight events. Using large-scale U.S. flight data (~70 million records) enriched with weather and operational features, we construct a two-hop aircraft rotation representation that captures delay carryover across consecutive legs.
We evaluate multiple modeling approaches, including gradient-boosted trees and sequence-based neural networks, under strict time-aware validation to ensure production realism. Results show that upstream delay propagation is the dominant predictive signal, with tree-based models achieving strong performance while remaining computationally efficient at scale.
Beyond prediction, this work establishes the foundation for AeroFlux, a real-time, network-aware system designed to evolve into a digital twin of the U.S. air transportation system, enabling forward simulation and proactive disruption management.

---

1 Introduction & Background

1.1 Motivation

Flight delays represent one of the most visible and persistent operational challenges in modern aviation. In 2025 alone, nearly 25% of U.S. domestic flights failed to arrive on time, marking

one of the worst on-time performance levels in over a decade [1]. At scale, these disruptions impose significant societal and economic costs—U.S. travelers collectively lost over 1.5 million hours (171 years) waiting on delayed flights [2], while delays across global air travel totaled millions of hours annually [3].

A single disruption rarely remains isolated. Instead, delays propagate through tightly coupled airline operations, affecting aircraft rotations, crew assignments, passenger connections, and airport congestion. Empirical studies of the U.S. air transportation network show that primary delays—caused by weather, congestion, or operational issues—can cascade and amplify across the network, leading to systemic disruptions that impact a significant portion of flights [4]. More recent research further demonstrates that accurately modeling delay propagation and absorption dynamics is essential for improving prediction performance and operational resilience [5].

Understanding the mechanisms of delay propagation is therefore critical for improving airline efficiency, reducing operational costs, and enhancing passenger reliability. As air traffic demand continues to grow and systems become increasingly interconnected, the ability to model and predict these cascading effects represents a key challenge—and opportunity—for modern aviation analytics.

1.2 Related Works

This work introduces a propagation-aware machine learning framework that models flight delays as a function of upstream aircraft state rather than isolated events. By incorporating a two-hop aircraft rotation structure, enforcing strict pre-departure feature availability, and training on multi-year, large-scale data (~70M flights), the approach produces both operationally realistic and high-performing predictions. It further serves as the foundation for AeroFlux, a system designed to evolve from static prediction into a real-time, network-aware digital twin of the U.S. airspace.

1.2.1 Time of Day Impacts

Flight delays exhibit strong temporal patterns, with peak congestion typically occurring during morning and evening "banking" periods at major hubs. Studies using Bureau of Transportation Statistics data consistently show that delays increase throughout the day due to cumulative disruption effects and reduced recovery buffers [6]. Prior work in transportation analytics confirms that departure time is one of the most statistically significant predictors of delay due to its correlation with demand surges and network congestion [7].

1.2.2 Delay Propagation

Delay propagation has been widely studied as a network phenomenon where disruptions spread through aircraft rotations and airport interactions. Foundational work by [4] demonstrates that primary delays can cascade across the U.S. air transportation network, amplifying system-wide disruption. Subsequent research modeled these dynamics using complex network theory, showing how disruptions propagate through tightly coupled flight schedules [8]. More recent work highlights that incorporating upstream flight state — including delay absorption capacity at intermediate airports — significantly improves predictive performance, reinforcing the importance of modeling propagation dynamics explicitly [5].

1.2.3 Quick Survey of Common Flight Delay Models

Machine learning approaches to flight delay prediction commonly include tree-based models such as XGBoost and deep learning models such as Long Short-Term Memory (LSTM) networks. XGBoost, introduced by [9], is widely favored for large-scale tabular data due to its strong performance, interpretability, and computational efficiency. LSTMs [10] are used to capture temporal dependencies across sequential flight events, making them a natural fit for rotation-chain modeling. Benchmark evaluations on structured tabular data consistently show that tree-based ensembles remain state-of-the-art over deep learning models, with deep learning offering advantages primarily when temporal or graph-based relationships are explicitly modeled [11].

More recent work has explored advanced architectures including Graph Neural Networks (GNNs) for modeling airport connectivity and network effects [12], Spatio-Temporal Graph Convolutional Networks (ST-GCN) for capturing joint spatial and temporal dependencies [13], Transformer-based sequence models applied to air traffic trajectory and flow prediction [14], [15], and attention-based multi-agent models for capturing interactions in air traffic systems [16].

1.2.4 Digital Twins and Real-Time Monitoring

Digital twins in aviation aim to create continuously updated virtual representations of physical systems for monitoring and simulation. Foundational frameworks in digital twin design emphasize the integration of live telemetry, predictive modeling, and simulation as the basis for proactive decision-making in complex physical systems [17], [18]. In the aviation domain, the Federal Aviation Administration's System Wide Information Management (SWIM) program represents a significant step toward real-time data integration for air traffic management and disruption response [19]. Recent work in cyber-physical systems reinforces that this combination of live data and predictive intelligence forms the foundation for next-generation intelligent transportation systems — a direction that directly motivates the AeroFlux architecture described in this paper.

Several operational systems now demonstrate this capability in practice. NASA Ames has developed a Digital Twin Simulator of the National Airspace System (NAS) that creates a virtual copy of the live NAS, ingesting real-time SWIM data to enable side-by-side simulation of proposed changes against actual operations [20]. At the industry level, Airspace Intelligence's Flyways AI platform applies a 4D airspace simulation built on live FAA SWIM feeds to predict demand-capacity imbalances and recommend optimized traffic management initiatives before disruptions materialize [21]. Similarly, Mosaic ATM's Alternate Route Availability Tool (ARAT), developed in collaboration with NASA, integrates real-time weather and TMI data with machine learning models to provide actionable alternate routing recommendations for air traffic controllers [22]. In the research literature, Liu et al. demonstrate an L3-level airport digital twin for flight ground service delay diagnosis that connects real-time operational data to simulation, enabling timely identification and propagation analysis of delays that offline models cannot achieve [23].

Recent work in cyber-physical systems reinforces that this combination of live data and predictive intelligence forms the foundation for next-generation intelligent transportation systems — a direction that directly motivates the AeroFlux architecture described in this paper.

1.3 Research Questions

This study seeks to answer the following questions:

1. To what extent does upstream aircraft delay propagation influence downstream flight delay outcomes?
2. Which features — including operational, weather, and network-based variables — provide the strongest predictive signal under realistic pre-departure constraints?
3. How do different machine learning approaches (tree-based vs. sequence-based) perform on large-scale, temporally structured aviation data?
4. Can a propagation-aware modeling framework serve as a foundation for real-time, network-level prediction and simulation systems in aviation?

1.4 What This Research Contributes

This research contributes a propagation-aware machine learning framework for flight delay prediction that explicitly models delay transmission across aircraft rotations. By introducing a two-hop representation of upstream flight state and enforcing strict pre-departure feature availability, the approach captures the operational dynamics underlying delay formation while remaining suitable for real-world deployment.

The study demonstrates the effectiveness of this framework at scale, training and evaluating models on approximately 70 million U.S. flight records using time-aware validation. Results show that delay propagation is the dominant predictive signal, significantly outperforming models that treat flights independently.

Beyond predictive performance, this work establishes the architectural foundation for AeroFlux — a real-time, network-aware system designed to evolve into a digital twin of the national airspace, enabling forward simulation, disruption propagation analysis, and proactive operational decision-making.

The ultimate objective of this work is to move beyond static prediction and toward dynamic, system-level reasoning in aviation operations. By combining propagation-aware modeling with real-time data integration and simulation capabilities, this research aims to enable predictive intelligence systems that can anticipate, propagate, and mitigate disruptions across the air transportation network. This vision is realized through AeroFlux, which represents a step toward a fully operational digital twin of the U.S. airspace.

2 Data

2.1 Data Sources & Pipeline

2.1.1 Data Pipeline

For each year of processing (2015-2025) the pipeline integrates three independent data streams — Bureau of Transportation Statistics (BTS) on-time performance records, NOAA Global Surface Hourly weather observations, and airport reference dimensions — into a unified analytical dataset suitable for delay prediction modeling. This process constitutes a data fusion framework, in which heterogeneous data sources are aligned and combined across temporal, spatial, and operational dimensions to enhance the representational richness of the dataset. Figure 1 provides a schematic overview of each stage, and the sections below describe each stage in turn. To run the pipeline or see how the code works see `data_pipeline` in the code base. Extensive use of Python Polars features were used to help with data processing efficiency.

2.1.2 Data Sources

The pipeline integrates three independent data streams: Bureau of Transportation Statistics (BTS) on-time performance records, NOAA Global Surface Hourly weather observations, and airport reference dimensions. Flight performance data are sourced from the BTS Transtats PREZIP endpoint, which publishes monthly on-time reporting files for all U.S. carriers, providing flight-level records including scheduled and actual departure and arrival times, delay durations, cancellation indicators, and delay-cause variables [6]. Weather observations are obtained from the NOAA NCEI global-hourly dataset via a REST API, providing sub-hourly surface measurements — temperature, wind speed, wind direction, and ceiling height — joined to flights using a backward-looking as-of temporal join [24]. Airport and station reference data are drawn from the OurAirports CSV registry and the NOAA Integrated Surface Database

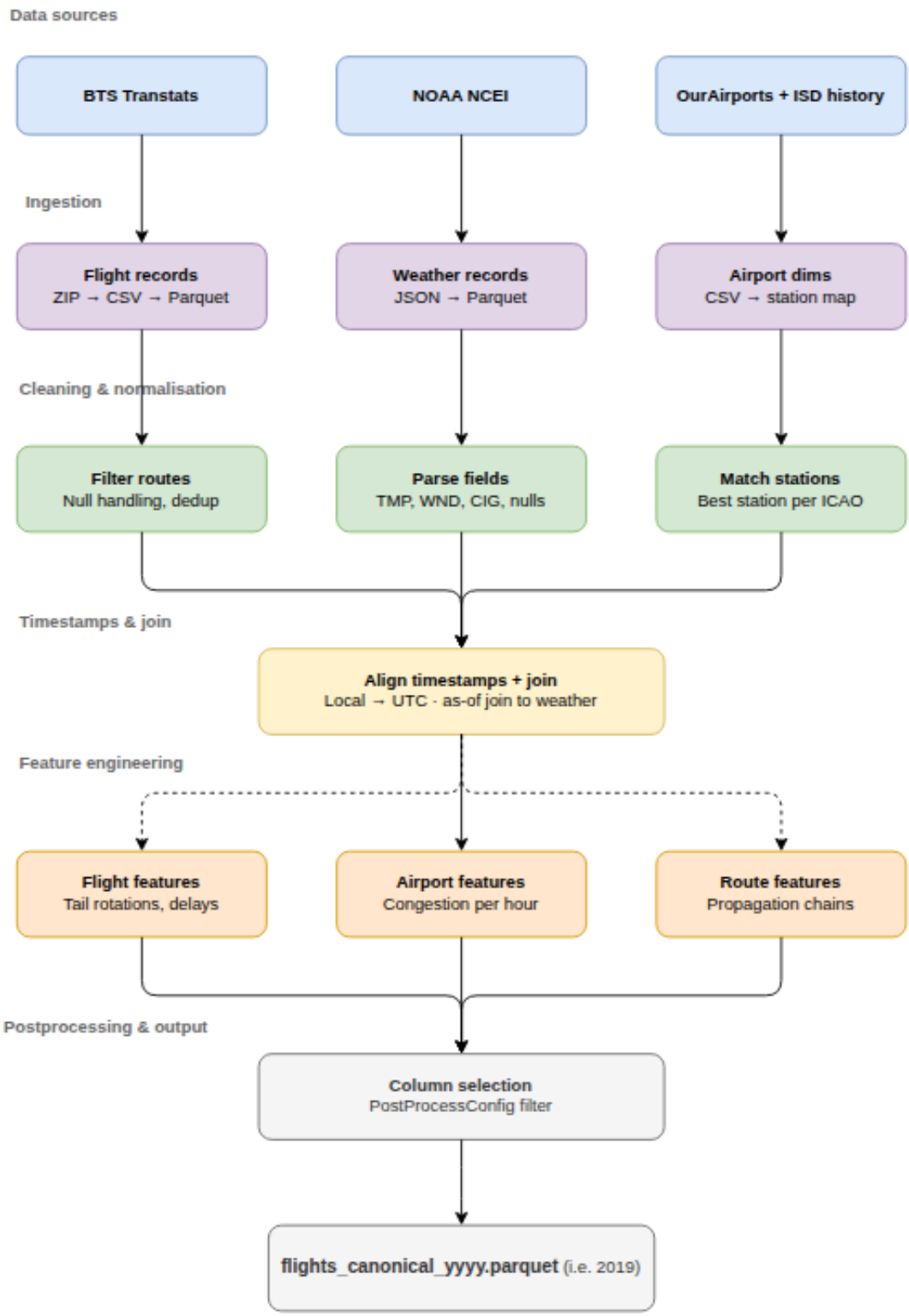

Figure 1: Overview of the flight delay data pipeline, from raw data sources through ingestion, cleaning, temporal alignment, feature engineering, and final output.

(ISD) station history file, supplying the geographic coordinates and ICAO codes required to link flight records to their corresponding weather stations [25], [26].

FAA aircraft registry variables — including number of seats, engine count, and aircraft type — were initially excluded due to a temporal mismatch between the available 2025 registry snapshot and the historical flight data. However, because these physical characteristics are not meaningfully time-dependent, they were reintroduced as an additional feature engineering step during the ML pipeline and form part of the `xgb_full_aircraft` feature set.

Table 1 summarises all primary data sources used in the pipeline.

Table 1: Primary data sources used in the flight delay pipeline.

Source	Description	URL
BTS Transtats — On-Time Performance	Monthly carrier on-time performance records; primary source of flight departure, arrival, delay, and cancellation data.	transtats.bts.gov
NOAA NCEI — Integrated Surface Database (ISD)	Global hourly surface weather observations from 14,000+ stations spanning 1901–present; used for station metadata and weather-to-flight spatial joins.	ncei.noaa.gov/products/land-based-station/integrated-surface-database
NOAA NCEI — Global Hourly Data Search	Programmatic API access to the NCEI global-hourly subset; supports station search and date-range subsetting.	ncei.noaa.gov/access/search/data-search/global-hourly
OurAirports — Open Data Downloads	Airport reference files with IATA/ICAO codes, coordinates, and metadata; GitHub-hosted daily dumps used as the canonical source.	ourairports.com/data
FAA Aircraft Registry	Aircraft physical characteristics including seat count, engine count, and aircraft type; 2025 snapshot used for time-invariant features in the xgb_full_aircraft feature set.	registry.faa.gov/aircraftinquiry

2.1.3 Ingestion

Each data stream is ingested independently and cached to Parquet before any downstream processing begins. This decouples the expensive download and format-conversion steps from feature engineering, allowing the pipeline to be re-run without re-fetching raw data.

- **BTS flight records** are downloaded as monthly ZIP archives from the BTS Transtats PREZIP endpoint using an HTTP session with exponential backoff (up to 6 retries, base

delay of 2 seconds). Each archive is extracted, parsed into a Polars DataFrame with explicit schema inference, and serialized to Parquet. Empty strings, `"NA"`, and `"NULL"` values are treated as nulls at parse time.

- **Weather observations** are retrieved from the NOAA NCEI REST API in station-month chunks of 5 stations per request, returned as JSON, and persisted to monthly Parquet cache files. A progressive fallback strategy handles malformed or truncated responses by splitting the date window from 15-day to 7-day to 3-day to 1-day chunks, and finally retrying each station individually if necessary.
- **Airport and station reference data** are loaded from CSV (OurAirports and NOAA ISD history) and held in memory as dimension tables for spatial and timezone joins downstream.

2.1.4 Cleaning and Normalization

Flight records with cancellations, diversions, or missing critical timestamps are removed, and duplicate records are eliminated. Numeric delay fields (`DepDelay`, `ArrDelay`) and their binary indicators (`DepDel15`, `ArrDel15`) are null-filled to zero for non-cancelled flights. Cancellation and diversion flags are cast to binary integers.

Weather records undergo field-level parsing to extract structured values from NOAA's packed string format:

- `TMP` → temperature in °C (integer divided by 10)
- `WND` → wind direction (degrees) and wind speed (m/s, divided by 10)
- `CIG` → ceiling height in meters

Observations where all three primary meteorological fields are null are dropped. Records are deduplicated on `(station, valid_ts)`, keeping the last observation per timestamp.

Airport-to-station mapping is constructed by matching each BTS airport to its nearest ISD weather station using a nearest-neighbour procedure over geodetic distance. Timezone resolution uses the `timezonefinder` library, with a fallback to a longitude-based UTC offset estimate for airports that cannot be resolved to a canonical IANA zone. For this study there were only 2 stations out of 360 that could not be located.

2.1.5 Timestamp Alignment and Temporal Join

Correct temporal alignment is the most consequential and error-prone step in the pipeline. The process involves three stages:

1. Local timestamp construction. BTS departure and arrival times are stored as four-digit integers in local airport time (e.g., `1435` = 14:35). These are zero-padded and parsed into naive

local datetimes using the flight date. A known edge case arises when a flight departs late at night and arrives after midnight. For example, a flight with a scheduled departure of **23:45** and arrival of **00:30** will have both times recorded as integers on the same `FlightDate`:

```
CRSDepTime = 2345
CRSArrTime = 0030
```

A naive parser would read the arrival as earlier than the departure on the same date. The pipeline detects this condition — where `arr_ts < dep_ts` — and adds one calendar day to the arrival timestamp, correctly placing it on the following date.

2. UTC projection. All local datetimes are converted to UTC using `ZoneInfo`-based offsets for both flight timestamps and NOAA weather observation timestamps. Any timestamp that cannot be resolved — due to a missing timezone or conversion error — is set to null and excluded from downstream joins. A one-hour offset error at a UTC−5 airport, for example, would cause the as-of join to select weather observations from the wrong side of a frontal passage, corrupting both meteorological predictors and propagation-chain features.

3. As-of join. Once both datasets share a common UTC timeline, flight records are joined to weather observations using a backward-looking as-of join: for each departure or arrival event, the most recent weather observation at the corresponding station that precedes the event time is selected, within a 2-hour tolerance window. Observations outside the tolerance are left as null rather than matched to a stale reading. Separate as-of joins are performed for the origin station at departure time and the destination station at arrival time, yielding paired meteorological snapshots for both endpoints of every flight.

For example, consider a flight departing ATL at **14:32 UTC**. The pipeline looks backward through ATL's weather observations and selects the most recent one recorded before that time — say, a station report at **14:20 UTC**. That 12-minute-old observation is attached to the flight as its departure weather snapshot.

If the most recent observation is older than 2 hours — for instance, the last report was at **11:45 UTC** — the weather fields are left as `null` rather than attached, since a reading that stale is unlikely to reflect actual conditions at departure.

```
ATL weather observations:
  11:45 UTC -> temp: 18°C, wind: 12 m/s   <- too old (> 2h), not used
  14:20 UTC -> temp: 22°C, wind:  8 m/s   <- selected (most recent before 14:32)
  14:55 UTC -> temp: 23°C, wind:  7 m/s   <- in the future, not used

Flight departs ATL at 14:32 UTC
  dep_temp_c          = 22.0
  dep_wind_speed_m_s  = 8.0
```

The same process is repeated independently for the arrival airport at the flight's arrival time, yielding a second weather snapshot for the destination.

This approach correctly handles the irregular cadence of surface weather reporting — naive equi-joins on rounded timestamps would systematically introduce look-ahead bias or large temporal gaps.

2.1.6 Postprocessing and Output

After feature engineering, the pipeline performs a final cleanup pass before writing outputs. Because the fully enriched dataset contains hundreds of columns — raw BTS fields, intermediate join keys, duplicate timestamp formats, and processing artifacts — only the modeling-relevant subset is retained. This selection is driven by a configured column list; any requested column not present in the data is logged as a warning and skipped.

Final outputs are written as annual Parquet files. The primary modeling dataset contains one row per flight with all joined weather, operational, and propagation features attached. Alongside it, separate Parquet tables are written for aircraft rotation sequences, propagation chains, airport-time aggregates, and route-time aggregates, which are consumed independently during the ML pipeline.

2.1.7 Feature Engineering

Three families of derived features are constructed from the aligned dataset, spanning flight-level identifiers, airport-level congestion context, and route-level propagation state. All features are strictly restricted to information available at or before scheduled departure time, ensuring no look-ahead bias.

2.1.7.1 Flight-Level Features

Flight-level features establish the core identity of each observation and capture schedule structure, operational outcomes, and temporal context. A unique flight identifier is constructed from carrier, tail number, origin, destination, and scheduled departure time. Aircraft rotation sequences link successive legs flown by the same tail number, forming the backbone of the propagation feature set.

2.1.7.2 Airport-Level Features

Airport-level features aggregate traffic volume, delay rates, and congestion indicators within configurable rolling time windows around each departure. These features capture the systemic load at both origin and destination airports independently of any individual aircraft's history.

2.1.7.3 Route-Level and Propagation Features

Route-level features encode delay transmission along aircraft rotation chains. A late-aircraft delay on an inbound leg is explicitly linked to the downstream departure it constrains, allowing the model to condition on upstream disruption state at prediction time. The two-hop propagation architecture, described below, operationalizes this mechanism as a structured feature family.

2.1.7.4 The Two-Hop Propagation Architecture

A central contribution of this framework is the explicit modeling of delay carryover across aircraft rotations through a **two-hop propagation structure**. A two-hop flight represents a single aircraft operating two consecutive legs on the same day, connected by a turnaround at an intermediate airport. When the first leg arrives late, the delayed aircraft becomes the inbound for the second leg, compressing or eliminating the scheduled ground time and propagating the delay forward.

Figure 2 illustrates this for tail N471AA operating BOS → ATL → MIA: a 52-minute departure delay on Leg 1 carries through the ATL turnaround and manifests as a 49-minute departure delay on Leg 2, despite no adverse weather at ATL.

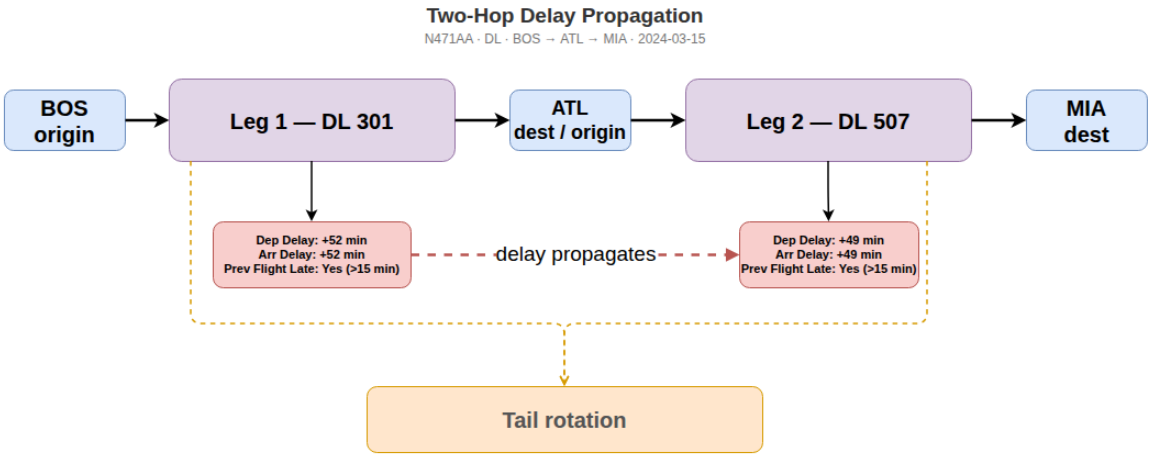

Figure 2: Two-hop delay propagation for tail N471AA (BOS → ATL → MIA, 2024-03-15). A 52-minute departure delay on Leg 1 propagates to a 49-minute delay on Leg 2 via the tail rotation, independent of weather conditions at the intermediate airport.

The turnaround buffer between consecutive legs is defined as:

$$\text{Turnaround}_i = \text{DepartureTime}_i - \text{PreviousArrivalTime}_i$$

A tight turnaround compresses the airline's ability to absorb inbound lateness, making this interval one of the most operationally informative features in the model. The propagation mechanism is captured through the `prev_arr_late_15` and `rotation_continuity_flag` columns, allowing the model to condition on upstream disruption state when predicting downstream delays.

Why two hops? The two-hop window balances propagation depth against feature complexity and temporal leakage risk. A single-hop representation misses the accumulated stress that builds across a multi-leg aircraft day, while deeper windows expand the feature space rapidly and increase the risk of incorporating state that was not yet observable at prediction time. Two hops provides a practical, well-motivated approximation of the delay carryover window most relevant to pre-departure prediction.

2.1.7.5 Feature Summary

Table 2 provides a complete inventory of all engineered features, organized by family.

Table 2: Complete engineered feature inventory organized by family.

Family	Feature(s)	Role
Flight identity	`FlightDate`, `Reporting_Airline`, `Tail_Number`, `Origin`, `Dest`, `route_key`, `flight_id`	Core identifiers; link feature tables and define route context
Schedule & timing	`CRSDepTime`, `DepTime`, `CRSArrTime`, `ArrTime`, UTC timestamp variants	Schedule adherence, temporal ordering, weather join keys
Delay targets	`ArrDelay`, `ArrDel15`, `DepDelay`, `DepDel15`, `Cancelled`, `Diverted`	Primary response variables and operational outcome filters
Surface movement	`TaxiOut`, `TaxiIn`, `Distance`, `ActualElapsedTime`	Operational complexity and route structure
Delay cause	`CarrierDelay`, `WeatherDelay`, `NASDelay`, `LateAircraftDelay`	Descriptive analysis and delay-driver interpretation

Family	Feature(s)	Role
Temporal / calendar	Departure hour, weekday, month, holiday indicators, `days_to_nearest_holiday`, `is_weekend`	Cyclic demand, seasonal patterns, calendar effects
Airport reference	Latitude, longitude, IATA/ICAO codes, timezone	Weather station mapping and timestamp normalization
Weather station ref.	NOAA station ID, station timezone, airport-to-station mapping	UTC conversion and weather-to-flight spatial joins
Weather	`dep_temp_c`, `dep_wind_speed_m_s`, `dep_wind_dir_deg`, `dep_ceiling_height_m`, `arr_*` variants	Meteorological conditions at departure and arrival; as-of joined within a 2-hour pre-departure window
Two-hop propagation	`prev1_arr_delay`, `prev1_dep_delay`, `prev1_arr_del15`, `prev1_dep_del15`, `prev2_arr_delay`, `prev2_dep_delay`, `prev2_arr_del15`, `prev2_dep_del15`, `prev1_turnaround_minutes`, `time_since_prev2_arrival_minutes`	Upstream delay state across the two most recent aircraft legs; dominant predictive signal
Rotation state	`rotation_continuity_flag`, `aircraft_leg_number_day`, `relative_leg_position`, `has_prev_leg`, `has_next_leg`, `is_middle_leg`, tight turnaround flags	Aircraft sequence position and vulnerability to upstream propagation
Aircraft metadata	`aircraft_age`, `aircraft_no_eng`, `aircraft_no_seats`, `aircraft_year_mfr`	Fleet composition signal; time-invariant FAA registry features
Airport-time context	Hourly flight counts, average/median delay, delay rate, cancellation rate, taxi metrics, rolling means over 1h/3h/6h	Airport congestion and short-horizon system state

Family	Feature(s)	Role
Route-time context	Hourly route counts, mean route delay, route delay rate, rolling means over 1h/3h/6h	Recent route-specific conditions and network-level delay tendencies

2.1.8 Sample Dataset

Table 3 shows three example records from the final modeling dataset, illustrating how flight operational data, weather observations, and propagation features are combined into a single row per flight.

Table 3: Three example records from the final modeling dataset.

Field	Record 1	Record 2	Record 3
FlightDate	2023-06-15	2023-06-15	2023-06-16
Reporting_Airline	WN	AA	UA
Tail_Number	N8731A	N3CDAA	N76502
Origin	ATL	DFW	ORD
Dest	ORD	LAX	EWR
CRSDepTime	0800	1430	0630
DepDelay	0	47	112
ArrDelay	-3	38	98
ArrDel15	0	1	1
dep_hour_local	8	14	6
dep_weekday_local	3	3	4
dep_month_local	6	6	6
dep_temp_c	21.4	34.2	14.8
dep_wind_speed_m_s	4.2	9.1	11.3
dep_ceiling_height_m	3200	9999	600
arr_temp_c	18.1	22.3	12.1
arr_wind_speed_m_s	6.7	3.4	14.2
arr_ceiling_height_m	2800	9999	450
prev1_arr_delay	0	52	89
prev1_arr_del15	0	1	1
prev1_turnaround_minutes	52	28	18
prev2_arr_delay	0	12	34
prev2_arr_del15	0	0	1
rotation_continuity_flag	1	1	1
aircraft_leg_number_day	2	3	4

Note Table 3 is for demonstration purposes.

2.2 Limitations in Data

Several structural limitations of the dataset should be acknowledged:

- **BTS delay categories** are self-reported across five broad buckets, obscuring compounding causes — a late aircraft delay may itself originate from a weather event several legs upstream.
- **Weather observations** are spatially approximate; ISD station measurements may not capture localized runway-level conditions or sub-hourly convective events.
- **Network effects** are only partially visible — route and airport aggregates cannot fully represent cascading disruptions originating at distant hubs beyond the two-hop propagation window.
- **Operational factors** including crew scheduling, gate conflicts, maintenance holds, and ATC staffing are absent from BTS records entirely.
- **FAA aircraft metadata** is drawn from a 2025 registry snapshot applied to historical data, introducing minor inconsistencies where aircraft were reconfigured or retired during the study period.

Despite these limitations, the dataset provides a strong foundation for large-scale delay prediction.

2.3 EDA

2.3.1 Distribution of Delays

2.3.2 Delays by Time of Day

2.3.3 Weather vs Delay

2.3.4 Delay Propagation

2.3.5 Centrality Metrics

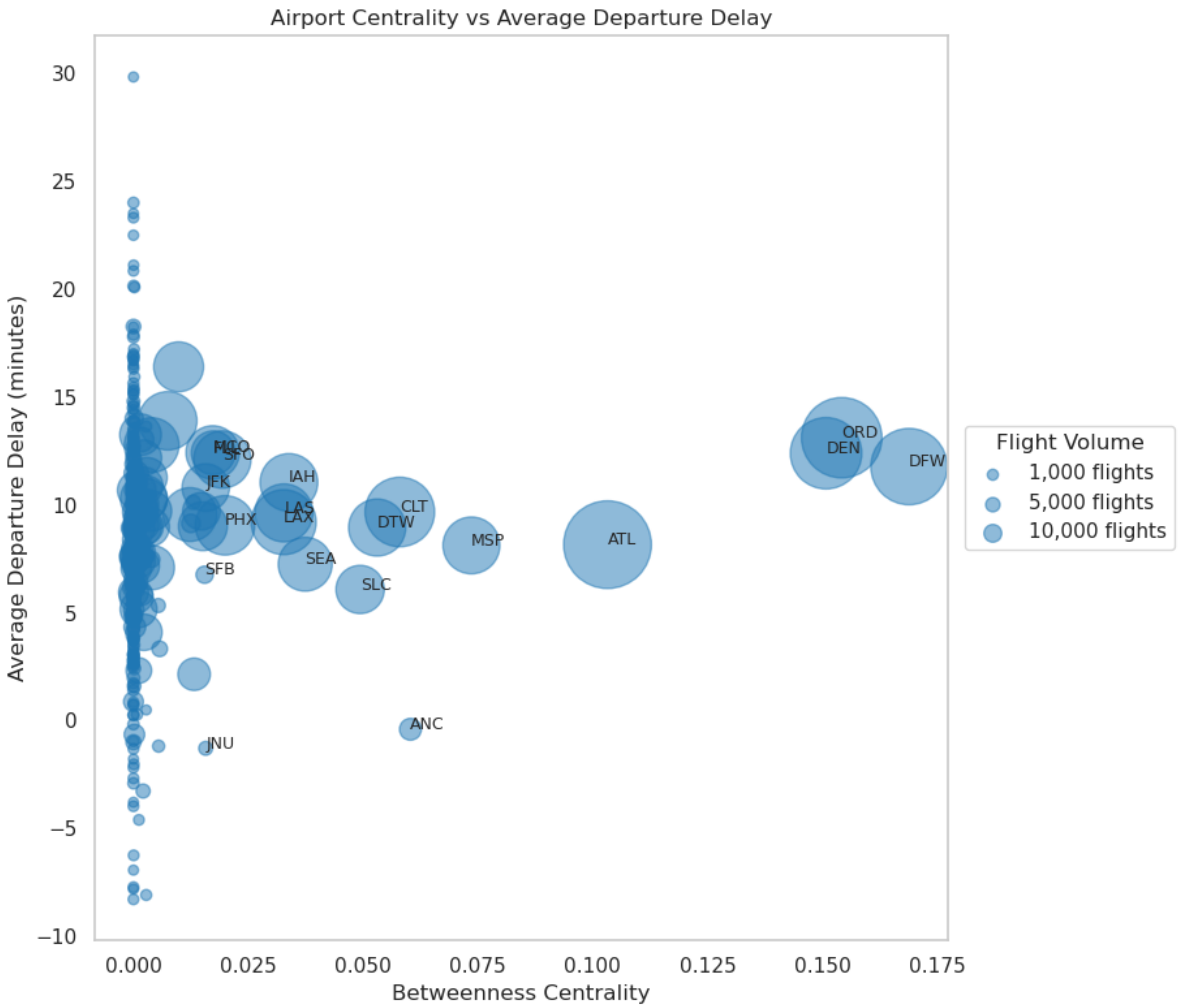

Figure 3: Centrality Metrics - Airport Bubble

3 Problem Formulation and Models

3.1 Two-Phase Modeling Approach

This study follows a two-phase modeling strategy to progressively develop and evaluate flight delay prediction methods. In Phase 1, we focused on a single-year dataset (2019) and implemented baseline models. This phase emphasized rapid iteration, feature validation, and establishing a foundational understanding of the problem.

In Phase 2, we expanded to a large-scale, multi-year dataset spanning approximately ten years of U.S. flight operations. This phase introduced more advanced modeling techniques, including gradient-boosted trees (XGBoost) and sequence-based neural networks (LSTM), along with a richer feature set incorporating weather, operational, and propagation-based variables. The transition from Phase 1 to Phase 2 reflects a shift from exploratory modeling to scalable, production-oriented system design.

3.2 Problem Formulation

We formulate flight delay prediction as two related supervised learning tasks over temporally ordered flight observations. For each flight i, let $\mathbf{x}_i \in \mathbb{R}^p$ denote the feature vector constructed from information available at or before prediction time.

We define two prediction targets:

$$Y_i^{(reg)} = \text{ArrDelay}_i$$

$$Y_i^{(cls)} = \begin{cases} 1 & \text{if ArrDel15}_i = 1 \\ 0 & \text{otherwise} \end{cases}$$

The classification task estimates the probability of a material arrival delay:

$$f_{cls}(\mathbf{x}_i) = P(Y_i^{(cls)} = 1 \mid \mathbf{x}_i)$$

The regression task estimates the expected delay magnitude in minutes:

$$f_{reg}(\mathbf{x}_i) = \mathbb{E}[Y_i^{(reg)} \mid \mathbf{x}_i]$$

In this study, classification and regression are modeled as separate but complementary supervised learning tasks for the XGBoost pipeline. The classification model estimates the probability of a material arrival delay, while the regression model estimates delay magnitude in minutes.

A joint multi-output formulation was explored separately in the LSTM architecture, where a shared sequence encoder feeds both a classification head and a regression head.

Crucially, all features are restricted to information available at or before prediction time, ensuring a realistic operational setting without look-ahead bias.

3.3 Phase 1: 2019 Baseline

3.3.1 Logistic Regression

We begin with a baseline logistic regression model.

$$P(Y_i = 1|X_i) = \frac{1}{1 + e^{-\eta_i}}$$

$$\eta_i = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + ... + \beta_p X_p$$

3.3.2 Ridge Logistic Regression

To address multicollinearity and improve stability we apply ridge regularization.

3.3.3 Basic Decision Trees

Decision trees provide interpretable classification rules.

3.3.4 Random Forest

Random forests improve predictive accuracy through ensemble learning.

3.3.5 K-Nearest Neighbors

KNN classifies flights based on nearby observations in feature space.

3.3.6 Unsupervised Analysis: Principal Component Analysis

PCA identifies dominant dimensions in the feature space.

3.3.7 Unsupervised Analysis: K-Means Clustering

3.4 Phase 2: Temporal Expansion and Model Scaling

In the final modeling training phase we used all our data. The 10 year run used over 61 million final training records, tuned it with rolling time-aware 5 fold cross-validation across historical years, then evaluated it on a fully unseen 2025 holdout test year containing 6.86 million flights. In total we are working with around 70 million flight records! The following sections will cover the details and how well the models performed.

3.4.1 Hardware

When working with larger data sets models were trained on a local workstation running Ubuntu 22.04.5 LTS (64-bit) with kernel version 6.8. The system is equipped with an AMD Ryzen 9 7900X processor featuring 24 logical cores and boost speeds up to 5.65 GHz, along with approximately 96 GB of system memory. The machine also includes both integrated AMD graphics and a discrete NVIDIA GPU, although model training was mostly conducted using CPU resources and GPU usage on 4 GB of VRAM didn't yield improved performance. The ML-Pipeline is still configured to run on GPU if desired. This hardware configuration provides substantial parallel processing capability and memory bandwidth, enabling efficient handling of large-scale data processing, feature engineering, and machine learning model training workflows.

3.4.2 LSTM Spatio-Temporal Model

3.4.2.1 Overview

A Long Short-Term Memory (LSTM) neural network was developed to model flight delay propagation using sequential aircraft rotation history. Unlike tree-based models that treat each flight independently, the LSTM learns temporal dependencies across prior legs operated by the same tail number — naturally representing how delays cascade through a rotation chain.

For a **2-hop propagation network**, the current flight is conditioned on the two most recent upstream legs:

$$JFK \rightarrow BOS \rightarrow ATL \rightarrow MIA$$

If the current flight is $ATL \rightarrow MIA$, then $JFK \rightarrow BOS$ and $BOS \rightarrow ATL$ form its sequential context.

3.4.2.2 Model Architecture

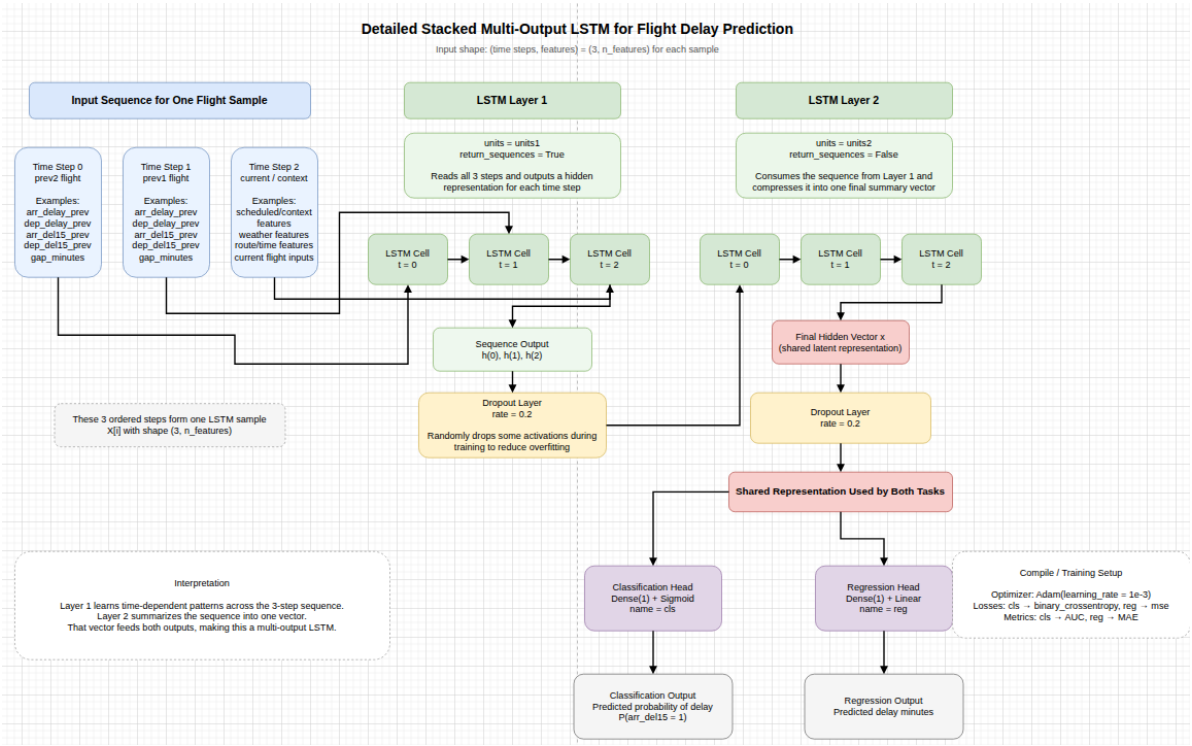

Figure 4: LSTM architecture for flight delay prediction. The network encodes a 3-timestep aircraft rotation sequence and produces two output heads: a binary delay classifier and a continuous delay regressor.

3.4.2.3 Input Representation

Each flight i is encoded as a 3D sequence tensor:

$$\mathbf{X}_i \in \mathbb{R}^{T \times F}$$

where $T = 3$ timesteps represent the **previous-2 leg**, **previous-1 leg**, and **current flight context**, and F is the number of engineered features per timestep. Across all training samples the full tensor is:

$$\mathbf{X} \in \mathbb{R}^{N \times T \times F} \quad \text{e.g.,} \quad (5{,}000{,}000,\ 3,\ 29)$$

Features at each timestep include prior arrival and departure delays, turnaround gap, distance, time-of-day, weekday, month, weather variables, route frequency, and congestion proxies.

3.4.2.4 Preprocessing and Scaling

Neural networks are sensitive to feature scale. The tensor is temporarily reshaped from (N, T, F) to $(N \cdot T,\ F)$, standardised via:

$$z = \frac{x - \mu}{\sigma}$$

then reshaped back, preserving sequence structure while ensuring stable gradient flow during training.

3.4.2.5 Sequence Encoder

The LSTM processes each timestep t and updates a hidden state:

$$\mathbf{h}_t = \mathrm{LSTM}(\mathbf{x}_t,\ \mathbf{h}_{t-1})$$

The final hidden state $\mathbf{h}_T$ serves as a compact representation of the full rotation history and is passed to both output heads. Each LSTM unit learns a temporal pattern from the input sequence; more units increase model capacity at the cost of overfitting risk.

Limiting the network to 2 hops ($T = 3$) provides a practical guard against exploding gradients, while the LSTM gating mechanism — maintaining separate long-term and short-term memory paths — mitigates the vanishing gradient problem inherent in standard RNNs.

A **dropout rate of 20%** was applied to LSTM outputs during training, randomly deactivating neurons to prevent the model from over-relying on any single feature or pattern.

3.4.2.6 Multi-Task Output Heads

The shared encoder feeds two independent output heads, enabling simultaneous binary classification and continuous regression from a single forward pass.

3.4.2.7 Classification Head

$$\hat{p}_i = \sigma(W_c\, \mathbf{h}_T + b_c)$$

Produces the probability that flight i is delayed by 15 minutes or more.

3.4.2.8 Regression Head

$$\hat{y}_i = W_r \, \mathbf{h}_T + b_r$$

Produces the predicted delay in minutes as a continuous value.

3.4.2.9 Loss Function

Both tasks are trained simultaneously using a single combined loss, so the weight updates from backpropagation account for both the classification and regression errors in every pass.

$$\mathcal{L} = \mathcal{L}_{cls} + \lambda \, \mathcal{L}_{reg}$$

where λ balances the contribution of each term. λ scales the regression loss so that neither task dominates training, since BCE and MSE naturally operate on different numerical scales.

Classification uses binary cross-entropy (BCE):

$$\mathcal{L}_{cls} = -\sum_i \left[y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i) \right]$$

This penalizes confident wrong predictions logarithmically, making it well-suited to binary delay outcomes.

Regression uses mean squared error (MSE):

$$\mathcal{L}_{reg} = \sum_i \left(Y_i^{(reg)} - \hat{Y}_i^{(reg)} \right)^2$$

Squaring the residuals penalizes large errors disproportionately, pushing the model to minimize severe mispredictions of delay magnitude.

3.4.2.10 Limitations and Model Status

Despite its architectural strengths, the LSTM was **dropped from the 10-year training run** due to:

- Higher memory footprint than gradient-boosted alternatives
- Slower training at scale ($N > 5$M samples)
- Reduced interpretability compared to tree-based models
- Non-trivial tensor preparation and scaling pipeline

The model is retained in this paper to document its design and to motivate future sequence-based extensions, which may include:

- Spatio-Temporal Graph Convolutional Networks
- Transformer sequence models
- Graph Attention Networks
- Hybrid LSTM + airport graph embeddings

3.4.3 XGBoost

A tuned Extreme Gradient Boosting (XGBoost) model was developed as the primary predictive model for flight delay forecasting.

XGBoost was selected for this task because gradient-boosted tree ensembles consistently achieve strong performance on large-scale structured tabular data. The method captures nonlinear operational interactions, tolerates heterogeneous feature scales without normalization, and remains computationally tractable at the scale of tens of millions of flight records. Unlike linear models, XGBoost can implicitly model feature interactions such as the combined effect of a tight turnaround window on an already-stressed aircraft — without requiring manual interaction terms.

The full modeling pipeline, from data splitting through final holdout evaluation, is described in the sections that follow.

3.4.3.1 Data Partitioning Strategy

To preserve temporal realism and eliminate any possibility of look-ahead bias, the dataset was partitioned **strictly chronologically**. Future observations were never permitted to influence model training or validation at any stage.

Table 4: Chronological data splits used across the full XGBoost pipeline.

Dataset	Years	Rows
Training	2015–2023	54,620,695

Dataset	Years	Rows
Validation	2024	6,947,566
Test (Holdout)	2025	6,857,845
Final Refit Train	**2015–2024**	**61,568,261**

The 2025 holdout set represents completely unseen flight operations and serves as the primary benchmark for reported final performance. No information from 2025 was available at any point during model development, hyperparameter search, or feature selection.

3.4.3.2 Feature Engineering

The final feature set for the production model (`xgb_full_aircraft`) contained **37 predictors** spanning five conceptually distinct categories. Feature selection was guided by domain knowledge of delay propagation mechanisms and validated empirically through cross-validation performance comparisons across feature subsets.

3.4.3.2.1 Schedule and Calendar Features

These features encode when a flight is scheduled, capturing peak-hour congestion effects, weekend travel patterns, holiday demand surges, and seasonal variation.

- `distance` — great-circle route distance in miles
- `crs_elapsed_time` — scheduled block time in minutes
- `dep_hour_local` — local departure hour (0–23)
- `dep_weekday_local` — day of week (Monday = 0)
- `dep_month_local` — calendar month
- `dep_time_bucket` — discretized departure time window (e.g., early morning, morning bank, afternoon, evening)
- `is_weekend` — binary flag for Saturday/Sunday operations
- `is_holiday` — binary flag for federal holiday proximity
- `days_to_nearest_holiday` — continuous proximity measure to the nearest holiday

3.4.3.2.2 Weather Features

Surface weather conditions at both the departure and arrival airports were joined from hourly ASOS/METAR records. Each flight received a pre-departure snapshot at the closest available observation prior to scheduled pushback.

Weather variables included:

- Temperature (°C)
- Wind speed (knots) and direction (degrees)
- Visibility (statute miles)
- Ceiling height (meters)
- Precipitation indicator

Example features: `dep_temp_c`, `dep_wind_speed_kt`, `arr_ceiling_height_m`, `arr_visibility_sm`.

3.4.3.2.3 Network Congestion Features

These features quantify operational load at both airports and on the specific route at departure time, capturing system-level congestion independent of any individual aircraft's history.

- `origin_flight_volume` — total scheduled departures at origin airport within a rolling time window
- `dest_flight_volume` — total scheduled arrivals at destination airport within a rolling time window
- `route_frequency` — historical frequency of delays on this origin–destination pair

3.4.3.2.4 Aircraft Rotation State Features

These features capture the operational history and physical characteristics of the specific aircraft assigned to the flight. Aircraft continuity — whether the same tail number operated the prior leg — is especially important for modeling mechanical and scheduling carryover.

- `aircraft_age` — years since aircraft manufacture
- `aircraft_no_eng` — number of engines
- `aircraft_no_seats` — certified seat capacity
- `aircraft_year_mfr` — year of manufacture
- `aircraft_leg_number_day` — how many legs the aircraft has already flown today
- `rotation_continuity_flag` — whether the inbound leg was operated by the same aircraft
- `relative_leg_position` — normalized position of this flight within the aircraft's daily rotation

3.4.3.2.5 Two-Hop Upstream Propagation Features

The most predictively powerful feature category encodes the delay state of the **same aircraft on its two most recent prior legs**. This two-hop window operationalizes the empirical reality that delays propagate forward through aircraft rotations: a late inbound flight becomes a late outbound flight.

Table 5: Two-hop upstream propagation features.

Feature	Description
`prev1_arr_delay`	Arrival delay (minutes) of the immediately preceding leg
`prev1_dep_delay`	Departure delay (minutes) of the preceding leg
`prev1_arr_del15`	Binary: was the preceding leg ≥ 15 min late on arrival?
`prev1_dep_del15`	Binary: was the preceding leg ≥ 15 min late on departure?
`prev2_arr_delay`	Arrival delay of the leg two hops prior
`prev2_dep_delay`	Departure delay of the leg two hops prior
`prev2_arr_del15`	Binary classification of two-hop-prior arrival delay
`prev2_dep_del15`	Binary classification of two-hop-prior departure delay
`prev1_turnaround_minutes`	Gate time between the inbound aircraft arrival and scheduled next departure
`time_since_prev2_arrival_minutes`	Elapsed time since the aircraft's second-prior arrival

Together, these features give the model visibility into the trajectory of delay accumulation along each aircraft's day, not merely its current scheduled state.

3.4.4 XGBoost Classification Model

The XGBoost classification model estimates the probability that a flight will experience a significant arrival delay:

$$\widehat{p}_i = P(Y_i^{(cls)} = 1 \mid \mathbf{x}_i) = \sigma\left(\sum_{m=1}^{M} f_m(\mathbf{x}_i)\right)$$

where $\sigma(z) = (1 + e^{-z})^{-1}$ and $M = 300$ gradient-boosted trees. where

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

is the logistic sigmoid function and $M = 300$ gradient-boosted trees. The binary decision rule applies a threshold of 0.5 i.e. the Classification Decision Rule:

$$\hat{Y}_i^{(cls)} = \begin{cases} 1 & \text{if } \hat{p}_i \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

3.4.5 XGBoost Regression Model

The XGBoost regression model directly predicts arrival delay in minutes as an additive ensemble of regression trees:

$$\hat{Y}_i^{(reg)} = \sum_{m=1}^{M} g_m(\mathbf{x}_i)$$

At each boosting iteration t, the model minimizes the regularized objective:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} l\Big(y_i,\, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)\Big) + \Omega(f_t)$$

where $l(\cdot)$ is the task-specific loss (log-loss for classification, squared error for regression) and the complexity penalty is:

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

with T denoting the number of leaves and w_j the leaf weights. This penalty discourages overly complex trees and provides implicit regularization against overfitting on the large but temporally structured dataset.

4 Model Evaluation Metrics & Comparitive Perfromance

4.1 ML Pipeline

- Add diagram of model training and evaluation process - move to modeling section
- Explain what the ML Pipeline is

4.2 Time-Aware Cross Validation

Standard random k-fold cross-validation is inappropriate for time-series data because it permits future observations to leak into training folds, producing optimistically biased estimates. Instead, a **rolling-origin expanding-window** cross-validation strategy was employed, where each fold trains on all years up to a cutoff and validates on the immediately following year.

$$\mathcal{D}_{\text{train}}^{(k)} = \{2015, \dots, t_k\}, \qquad \mathcal{D}_{\text{val}}^{(k)} = \{t_k + 1\}$$

The five folds used were:

Table 6: Rolling-origin cross-validation folds.

Fold	Training Years	Validation Year
1	2015–2018	2019
2	2015–2019	2020
3	2015–2020	2021
4	2015–2021	2022
5	2015–2022	2023

This design directly simulates production deployment conditions: the model is trained on all available history, then evaluated on the next operational year it has never seen. The expanding window ensures that later folds benefit from more training data, reflecting how a deployed model accumulates history over time.

4.3 Hyperparameter Search

A grid search was conducted over four candidate XGBoost configurations, each evaluated using the five-fold rolling cross-validation procedure. The search space covered the primary levers controlling model capacity and regularization:

- Number of trees (M)
- Maximum tree depth (d)
- Learning rate (η)
- Row subsampling ratio (s)
- Column subsampling ratio (c)
- Minimum child node weight
- L2 leaf weight regularization (λ)

4.3.1 Candidate Configurations

Table 7: Candidate hyperparameter configurations evaluated during grid search.

Config	Trees (M)	Depth (d)	LR (η)	Subsample (s)	Colsample (c)
`xgb_00`	250	4	0.05	0.8	0.8
`xgb_01`	250	4	0.05	0.8	1.0
`xgb_02`	300	6	0.05	0.8	0.8
`xgb_03`	400	6	0.03	0.8	0.8

4.3.2 Selected Configuration

The best-performing configuration was **`xgb_02`** (referred to as `xgb_03` in the production pipeline artifact). Its full parameter set is:

$$M = 300, \quad d = 6, \quad \eta = 0.05, \quad s = 0.8, \quad c = 0.8$$

Table 8: Winning XGBoost hyperparameter configuration selected from four candidate configurations.

Hyperparameter	Value
n_estimators	300
max_depth	6
learning_rate	0.05
subsample	0.8
colsample_bytree	0.8
min_child_weight	1
reg_lambda	1.0
tree_method	hist
device	cpu

The `hist` tree method was selected for computational efficiency on the large training dataset. The moderate depth of 6 and conservative learning rate of 0.05 balance model expressiveness with generalization, while subsampling at 80% for both rows and columns introduces stochasticity that reduces variance.

4.4 Cross-Validation Performance

4.4.1 Classification Performance (Mean Across 5 Folds)

Table 9: Cross-validation classification performance for the selected XGBoost model (xgb_03), mean and approximate standard deviation across 5 rolling folds.

Metric	Mean	Min	Max	Approx. SD
AUC	0.8357	0.8048	0.8525	0.0119
F1	0.5025	0.4122	0.5702	0.0395
Precision	0.7484	0.5696	0.8356	0.0665
Recall	0.3822	0.2977	0.4548	0.0393
Accuracy	0.8714	0.8532	0.9093	0.0140

4.4.2 Regression Performance (Mean Across 5 Folds)

Table 10: Cross-validation regression performance for the selected XGBoost model (xgb_03), mean and approximate standard deviation across 5 rolling folds.

Metric	Mean	Min	Max	Approx. SD
MAE (minutes)	18.09	17.12	18.89	0.44
RMSE (minutes)	40.86	35.07	45.49	2.61

> **Note**
>
> Standard deviations are approximated as $\mathrm{SD} \approx (\max - \min)/4$, a standard range-based estimate applicable when per-fold values are not stored. These values are conservative lower bounds on true variability.

4.4.3 Interpretation of Cross-Validation Stability

The cross-validation results reveal a model with **strong temporal robustness**. The most discriminative metric — AUC — exhibited the lowest variability (SD = 0.012), indicating that the model's ability to rank delayed flights above non-delayed flights remained stable across years that spanned a full decade of US aviation operations, including the COVID-19 disruption period (2020–2021) and the subsequent recovery years.

Stability was similarly high for overall accuracy (SD = 0.014) and mean absolute error (SD = 0.44 minutes), confirming that point predictions for delay magnitude did not drift substantially across validation years.

Larger variability in F1 (SD = 0.040), recall (SD = 0.039), and especially precision (SD = 0.067) reflects genuine annual heterogeneity in the delay-event distribution. Pandemic-era years exhibited structurally different delay patterns — reduced traffic volumes but elevated per-flight delay rates in some months — which affected the class-level precision-recall balance without substantially degrading rank-order discrimination. The RMSE variability (SD = 2.61 minutes) similarly reflects the influence of rare, operationally extreme years on tail-error statistics.

4.5 Final Refit and Holdout Evaluation

4.5.1 Refit Procedure

Following hyperparameter selection, the model was retrained on all pre-test data to maximize the information available for the final deployed model:

$$\mathcal{D}_{\text{final train}} = \{2015, \dots, 2024\}$$

The full refit dataset contained **61,568,261 rows**. For memory efficiency during the refit phase, a stratified **35% sample** (21,548,891 rows) was drawn, preserving the class distribution while reducing peak memory requirements. The final model was then evaluated on the completely held-out 2025 dataset.

4.5.2 Final Holdout Test Results (2025) - Post Tuning

4.5.2.1 Classification Performance

Table 11: Final holdout classification performance on unseen 2025 flight operations.

Metric	Value
AUC	0.8452
F1	0.5562
Precision	0.8029
Recall	0.4254
Accuracy	0.8486

4.5.2.2 Regression Performance

Table 12: Final holdout regression performance on unseen 2025 flight operations.

Metric	Value
MAE (minutes)	20.14
RMSE (minutes)	46.17

In summary:

$$\text{AUC} = 0.8452, \quad F_1 = 0.5562, \quad \text{MAE} = 20.14 \text{ min}, \quad \text{RMSE} = 46.17 \text{ min}$$

4.5.3 Interpretation of Final Results

Discrimination (AUC = 0.845). The model correctly ranks a randomly selected delayed flight above a randomly selected on-time flight 84.5% of the time on 2025 data. This represents a meaningful improvement over the cross-validation mean AUC of 0.836, suggesting that the model benefited from the expanded 2015–2024 refit training set. An AUC exceeding 0.84 is operationally strong for a binary event as dependent on real-time conditions as flight delay.

Precision (80.3%). When the model predicts a delay, it is correct more than four times out of five. This high precision makes the model practically useful for triggering downstream operational actions — passenger notifications, gate reassignments, crew advisory alerts — where false alarms carry real cost.

Recall (42.5%). The model recovers approximately 42.5% of all true delay events at the default 0.5 classification threshold. This relatively conservative recall reflects the class imbalance inherent in flight delay data: on-time flights substantially outnumber delayed flights in the dataset, and the default threshold does not optimize for recall. For use cases requiring higher sensitivity — such as proactive wide-net disruption planning — the decision threshold can be lowered to improve recall at the cost of some precision.

Regression error (MAE = 20.1 min, RMSE = 46.2 min). The mean absolute prediction error of approximately 20 minutes is reasonable given the inherent unpredictability of delay magnitude, which is influenced by late-breaking weather, ATC decisions, and mechanical events not fully captured in the pre-departure feature window. The substantially higher RMSE reflects the model's tendency to underestimate severe delays (> 2 hours), consistent with the general difficulty of predicting tail events in heavy-tailed delay distributions.

4.6 Feature Importance Analysis

Table 13: Top predictive features by XGBoost gain-based importance score.

Rank	Feature	Importance	Description
1	prev1_arr_del15	0.5106	Binary: inbound aircraft arrived 15 min late (immediate prior leg)
2	prev1_dep_del15	0.1247	Binary: inbound aircraft departed 15 min late (immediate prior leg)
3	prev1_arr_delay	0.1135	Continuous arrival delay in minutes of the immediately prior leg
4	dep_hour_local	0.0277	Local departure hour of day
5	dep_time_bucket	0.0267	Discretized departure time window
6	prev1_turnaround_minutes	0.0262	Gate turnaround time between inbound arrival and scheduled next departure

The feature importance profile is unambiguous: **delay propagation through aircraft rotations is the dominant predictive signal**, accounting for over 75% of total model gain across the top three features alone. The binary flag `prev1_arr_del15` — whether the inbound leg arrived at least 15 minutes late — contributes over half of all model importance (0.511), confirming that the single most informative question about any outbound flight is: *was the same aircraft delayed on its way in?*

This finding has important operational implications. It validates the two-hop propagation feature engineering strategy and suggests that real-time monitoring of upstream aircraft state is far more valuable for delay prediction than any static schedule or network congestion feature. Schedule timing features (`dep_hour_local`, `dep_time_bucket`) appear in the top six but at an order of magnitude less importance than propagation features.

Aircraft metadata variables (`aircraft_no_seats`, `aircraft_year_mfr`, `aircraft_age`, `aircraft_no_eng`), while individually modest in importance, collectively provided measurable incremental lift in controlled ablation experiments comparing the `xgb_full_aircraft` feature set to the `xgb_2hop_propagation` subset alone. This suggests that fleet composition and aircraft utilization patterns carry weak but real signal about delay susceptibility — potentially through maintenance scheduling effects or route assignment patterns associated with specific aircraft types.

4.7 Feature Set Comparison

To isolate the contribution of each feature category, four nested feature configurations were evaluated under identical cross-validation conditions:

Feature Set	Description
`xgb_schedule`	Schedule and calendar features only
`xgb_context`	Schedule + weather + network congestion
`xgb_2hop_propagation`	All features except aircraft metadata
`xgb_full_aircraft`	Complete 37-feature set including aircraft metadata

The fully enriched `xgb_full_aircraft` model was the best overall performer across both classification and regression metrics, confirming that each feature category contributes incrementally. The largest single performance gain was observed when two-hop propagation features were added to the context feature set, consistent with their dominant importance scores. Aircraft metadata provided a smaller but consistent additional improvement.

4.8 Computational Performance

The complete XGBoost pipeline — encompassing data loading, FAA registry enrichment, rolling cross-validation across five folds, hyperparameter evaluation across four configurations, final refit, and artifact generation — completed in approximately **1,035 seconds (~17.3 minutes)** on CPU hardware using 8 parallel worker cores.

This compares favorably against deep learning alternatives (e.g., LSTM-based models requiring GPU hardware and substantially longer training times) and demonstrates that XGBoost remains a practical choice for production-scale aviation delay forecasting without specialized compute infrastructure.

4.9 Model Comparison

4.10 Summary

The tuned XGBoost model achieved strong predictive performance on completely unseen 2025 US domestic flight operations after hyper parameter tuning:

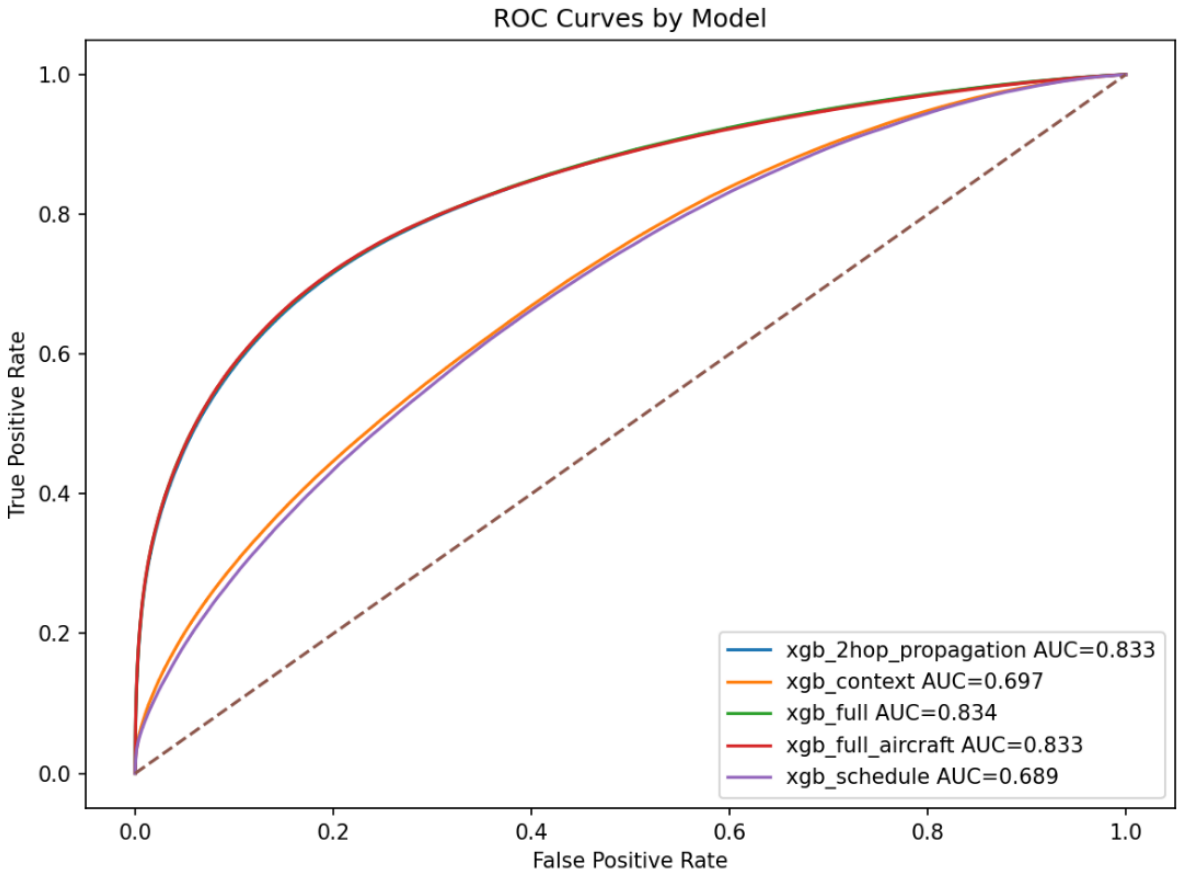

Figure 5: ROC curves for model varients on the 2025 holdout evaluation set - Before Tuning.

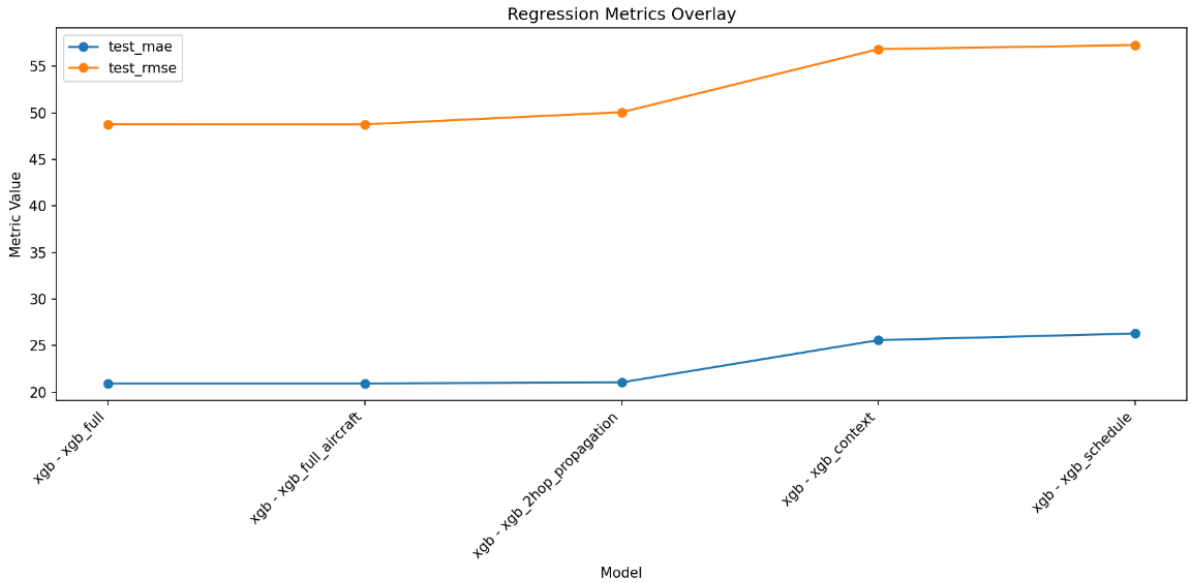

Figure 6: Regression metrics for model varients on the 2025 holdout evaluation set - Before Tuning.

$$\text{AUC} = 0.845, \quad F_1 = 0.556, \quad \text{Precision} = 0.803, \quad \text{MAE} = 20.1 \text{ min}$$

The results confirm that **aircraft rotation propagation is the primary mechanism underlying flight delay formation** at the national scale: the single most predictive feature — whether the same aircraft arrived late on its immediately prior leg — contributed over 50% of total model gain. Schedule, weather, and network congestion features provided meaningful secondary signal, while aircraft metadata added modest but consistent incremental improvement.

The rolling-origin cross-validation strategy yielded stable performance estimates across five temporal folds spanning a decade of operational history, with low AUC variance (SD = 0.012) confirming that the model learned transferable patterns rather than overfitting to any particular year's conditions.

Overall, XGBoost proved to be a computationally efficient, highly interpretable, and production-ready modeling approach for large-scale national flight delay prediction, providing a strong baseline against which more complex sequential architectures may be compared.

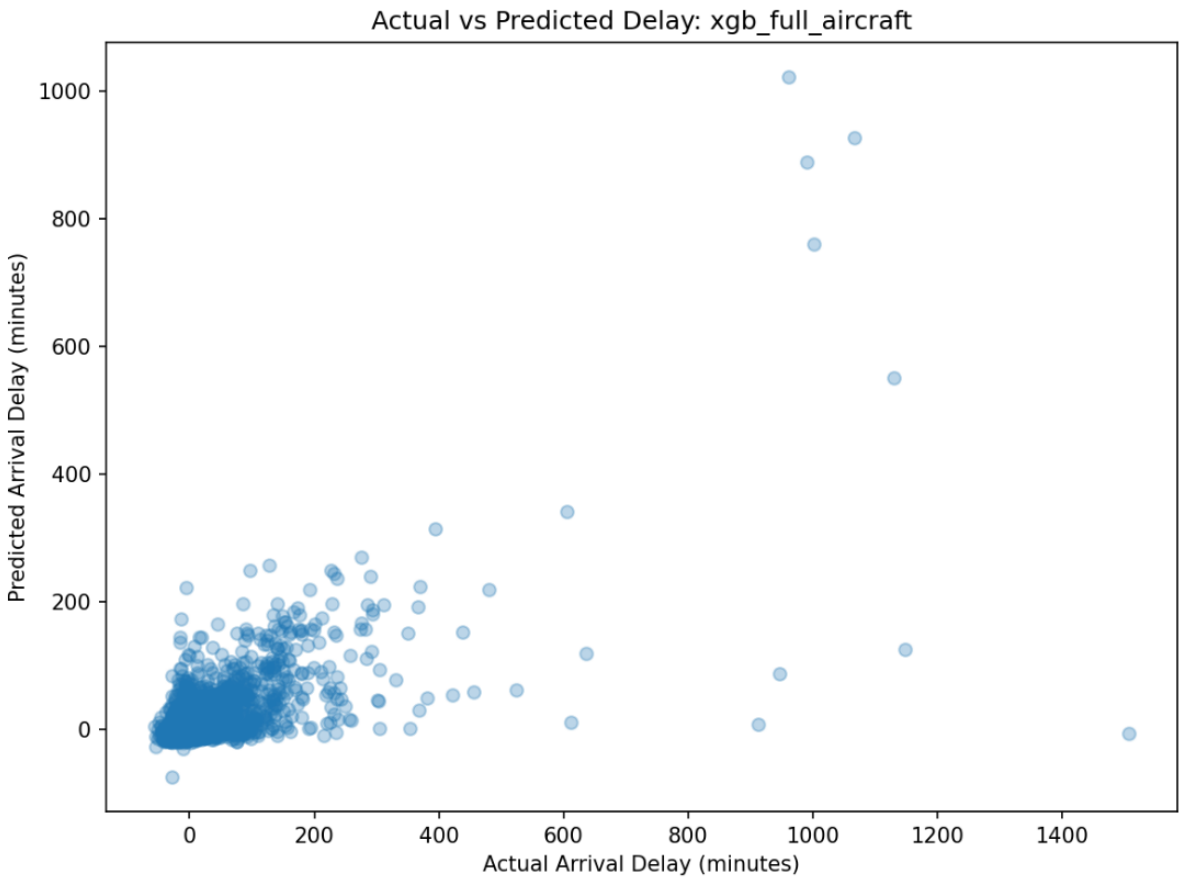

Figure 7: Actual versus predicted delay for the xgb_full_aircraft model on the 2025 holdout evaluation set - Before Tuning.

4.11 Operational Impact

Translating model performance metrics into operational terms requires applying the holdout classification results to the scale of actual flight operations. The 2025 holdout set contained **6,857,845 flight records**, of which approximately **23.4%** were labeled as delayed (`ArrDel15` = 1), yielding roughly **1,604,736 true delay events** in the evaluation period.

At the default threshold of 0.50, the model's precision of 0.803 and recall of 0.425 imply the following approximate operational outcomes across the holdout year:

Table 14: Approximate annual operational outcomes at the default threshold (0.50), derived from 2025 holdout classification results.

Outcome	Approximate Count	Note
Total flights evaluated	6,857,845	
True delay events (actual)	~1,604,736	23.4% base delay rate
Delay alerts issued (predicted positive)	~851,468	Precision × TP + FP
Correct alerts — True Positives	~681,613	Recall × true delay events
False alerts — False Positives	~166,951	Correctly flagged but flight was on time
Missed delays — False Negatives	~923,123	Delayed flights the model did not flag
Correct non-alerts — True Negatives	~5,086,158	On-time flights correctly left unflagged

> **Note**
>
> Counts are derived from reported precision (0.803) and recall (0.425) applied to the approximate true delay count. Verify against your confusion matrix outputs for exact figures.

(REVIEW THIS SECTION - What is here is just an example)

Several operational implications follow directly from these numbers.

Scale of actionable intelligence. The model issues approximately **851,000 delay alerts annually** — roughly 2,330 per day across the national network — of which over **680,000 are correct**. At this scale, even a modest per-alert value (e.g., $50 in avoided passenger reaccommodation costs or crew repositioning savings) implies substantial aggregate operational value.

Cost of false alarms. Approximately **167,000 false alerts** are issued annually. The operational cost of each false alarm depends on what action it triggers. For low-cost interventions such as informational passenger notifications, false alarm costs are minimal. For high-cost actions such as preemptive aircraft swaps, the false alarm rate justifies threshold adjustment upward toward 0.70 (see **?@tbl-threshold-sweep**).

Scale of missed delays. The model misses approximately **923,000 true delay events** annually — delays that occur without a prior model alert. The majority of these are likely short delays (15–30 minutes) driven by factors not well represented in pre-departure features, such as late ATC clearances or gate conflicts. A recall-optimized threshold would reduce this number at the cost of additional false alarms.

Regression precision in context. A mean absolute prediction error of 20.1 minutes, applied across the approximately 681,000 correctly identified delay events, means the model's delay magnitude estimates are typically within one connection window of the true delay. For airports with 30-minute minimum connection times, a 20-minute MAE is operationally meaningful for rebooking triage.

These figures underscore that the model is not a laboratory artifact — at national scale, even imperfect predictions at this precision level represent a substantial operational information advantage over uninformed scheduling assumptions.

4.12 Model Limitations

The 2-hop propagation framework was designed to capture delay carryover from the two most recent upstream aircraft legs. This was a practical and computationally efficient approximation of delay dynamics: While highly useful, this design imposes important structural limitations for both LSTM and XGBoost model approaches.

(REVIEW)

- Core Limitation: Finite Memory Window. The 2-hop model only observes two upstream legs.
- Hidden Accumulated Operational Stress. Aircraft systems often degrade gradually throughout the day.
- Same-Aircraft Only Perspective.
- The propagation design centered on aircraft tail-number continuity:
- No Explicit Airport Network Topology. The 2-hop model does not explicitly represent the airport graph:
- XGBoost can model nonlinear interactions, but it does not maintain hidden recurrent state.
- Feature Explosion Risk. As hops increase the feature space grows rapidly.
- Temporal Leakage Risk. Both approaches must ensure upstream features are truly available before prediction time.

5 Discussion

(REVIEW & ADD MORE)

6 Conclusion

(REVIEW & ADD MORE)

This study demonstrates how machine learning methods can be applied to understand delay dynamics…

Future work could extend this framework to larger airline networks and incorporate time-series modeling approaches…..etc.

7 Appendex

7.1 Introduction to AeroFlux: Real-Time Inference for Flight Delay Prediction

AeroFlux is a predictive intelligence system for U.S. domestic flight delays. Given a specific flight — identified by tail number, date, origin, and destination — it predicts whether that flight will arrive more than 15 minutes late and estimates the total delay in minutes.

Rather than treating each flight in isolation, AeroFlux models the full operational history of the aircraft: the two preceding legs, their on-time performance, turnaround time, cumulative delay buildup over the course of the day, and weather conditions at departure. This context-aware approach captures the cascading nature of flight delays in ways that single-flight models cannot.

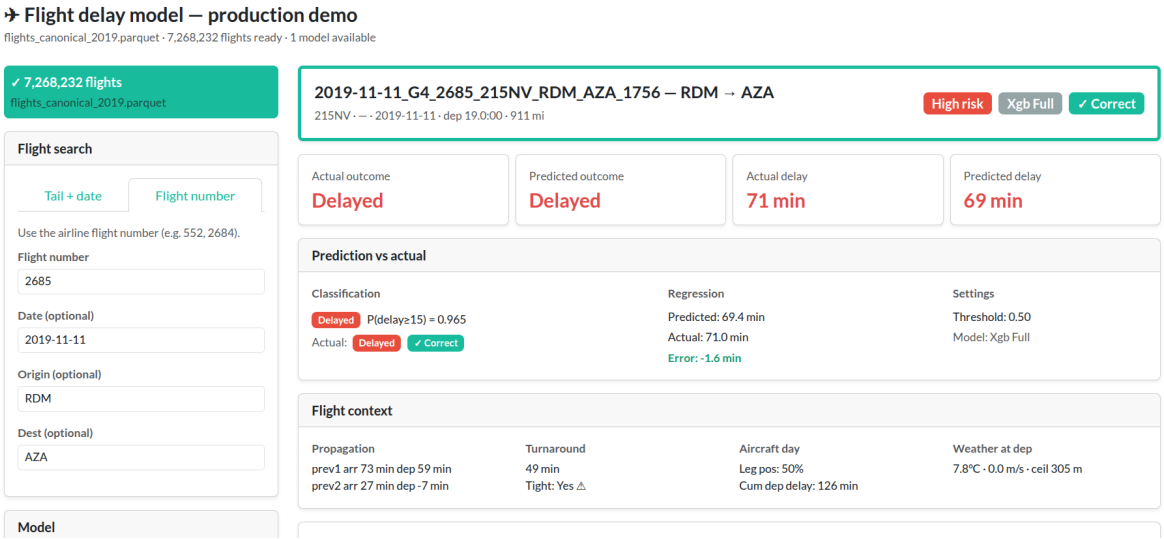

Figure 8: AeroFlux predictive intelligence system user interface

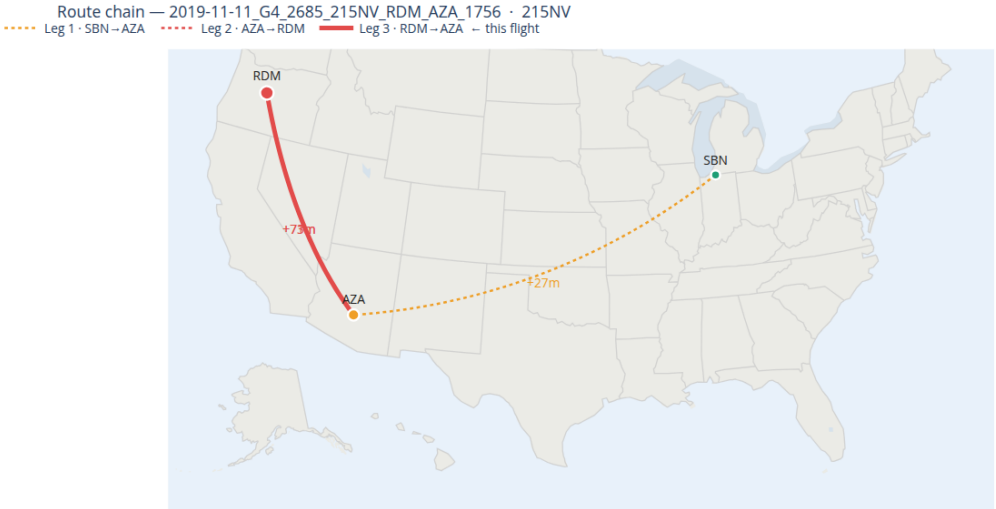

Figure 9: AeroFlux Flight Map

7.1.1 AeroFlux is the precursor to a digital twin

AeroFlux is not yet a digital twin but will soon be one. A true digital twin would ingest live operational data in real time, continuously update its state, and support simulation — answering questions like *what happens to the next four flights on this tail if this departure slips 45 minutes?*

AeroFlux is the immediate precursor. The machine learning pipeline, feature engineering, model inference layer, and interactive interface are all in place. What it currently lacks is the live data feed connecting it to real-time flight state. That connection is the primary objective for the next phase of development. The progression looks like this:

- **Phase 1 — Historical Proof of Concept** *(complete)* Baseline models trained, data and ML pipelines built, and an interactive prototype deployed. The foundation is in place: a working system that can ingest flight records, engineer operational context features, and produce interpretable delay predictions.
- **Phase 2 — Live Data & Comprehensive Modeling** *(next)* Connect to FAA or a commercial flight data API to replace historical lookups with real-time inference. Layer in live weather and social media feeds, and research more robust modeling architectures that better capture spatial-temporal dependencies across a flight day. Enhancing the UI with beautiful data visualizations.
- **Phase 3 — Network Simulation** Using Rust as a high-performance simulation engine, propagate predicted delays forward through the full rotation chain. Model downstream passenger connections, gate conflicts, and crew constraints — shifting AeroFlux from a single-flight predictor into a network-aware reasoning system.
- **Phase 4 — Digital Twin** A continuously updated, bidirectional model of the entire U.S. domestic flight network. Full scenario simulation: what happens to 400 downstream flights if a hub goes offline? What's the optimal recovery sequence after a ground stop? AeroFlux becomes a live mirror of the national airspace.
- **Beyond Phase 4 — The Intelligent Orchestration Layer** This is where AeroFlux transcends flight delays entirely and becomes a framework for reasoning about any system of moving objects in physical space. The ability to model intent, predict state, and propagate consequences through a network of interdependent agents is one of the most valuable capabilities in safety-critical systems — and nowhere is this more consequential than in avionics. Air Traffic Control operates at the edge of human cognitive limits, managing hundreds of aircraft simultaneously with razor-thin margins for error. A system that can anticipate conflicts before they materialize, surface the highest-risk situations in real time, and suggest resolution paths doesn't just improve efficiency — it fundamentally changes the error surface, shifting ATC from reactive management to proactive orchestration.

The architecture — real-time telemetry ingestion, predictive state modeling, network propagation, and scenario simulation — maps directly onto adjacent domains: urban air mobility fleets of autonomous air taxis, drone delivery networks optimizing thousands of concurrent last-mile routes, autonomous vehicle coordination across highway and urban grids, maritime vessel scheduling through congested port corridors, and satellite constellation management.

See AeroFlux here

Official AeroFlux App - Note: this domain might change in the future. If this happens notify the authors for more information.

8 Additional

Baseline Comparison - table of all models

Comparison to Prior Work: Prior studies typically report AUC values in the range of 0.75–0.82 for flight delay prediction tasks. The proposed propagation-aware framework achieves an AUC of 0.845 on a fully unseen 2025 dataset, suggesting that incorporating upstream aircraft state provides measurable predictive gains over traditional approaches.

Calibration / Probability Quality

Threshold Sensitivity Analysis Error Analysis - When does the model fail

4. Problem Formulation and Models 4.1 Problem Definition 4.2 XGBoost Models 4.3 LSTM Model 4.4 Training Strategy
5. Experimental Setup train/val/test split years dataset sizes hardware (optional but good) hyperparameters (brief)
6. Results 6.1 Performance - AUC, F1, MAE, RMSE 6.2 Baselines 6.3 Feature Analysis
7. Discussion
8. Limitations
9. Future Work (AeroFlux direction)
10. Conclusion

[1] USAFacts Team, "What are the best and worst airports and airlines for on-time performance?" https://usafacts.org/articles/what-are-the-best-and-worst-airlines-for-on-time-performance/; USAFacts, 2025.

[2] People Staff, "The world's most delayed airline in 2025 might surprise travelers." https://people.com/the-world-s-most-delayed-airline-in-2025-might-surprise-travelers-11870658; People, 2025.

[3] SkyRefund, "SkyRefund analysis reveals US travelers lost 1.5 million hours to flight delays in 2025 — and what caused them." https://www.prnewswire.com/news-releases/skyrefund-analysis-reveals-us-travelers-lost-1-5-million-hours-to-flight-delays-in-2025--and-what-caused-them-302675101.html; PR Newswire, Jan. 2026.

[4] P. Fleurquin, J. J. Ramasco, and V. M. Eguiluz, "Systemic delay propagation in the US airport network," *Scientific Reports*, vol. 3, no. 1, Jan. 2013, doi: 10.1038/srep01159.

[5] J. Zhou, "Integrating delay-absorption capability into flight departure delay prediction." 2025. Available: https://arxiv.org/abs/2512.08197

[6] Bureau of Transportation Statistics, "Airline on-time performance data." https://transtats.bts.gov/; U.S. Department of Transportation, 2025.

[7] J. J. Rebollo and H. Balakrishnan, "Characterization and prediction of air traffic delays," *Transportation Research Part C*, vol. 44, pp. 231–241, 2014.

[8] J. Li, N. Chen, and J. Zhang, "Modeling and simulation of flight delay propagation based on complex networks," *Physica A*, vol. 444, pp. 102–112, 2016.

[9] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," *Proceedings of the ACM SIGKDD*, 2016.

[10] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[11] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on tabular data?" in *Advances in neural information processing systems (NeurIPS)*, 2022. Available: https://arxiv.org/abs/2207.08815

[12] L. Zhao *et al.*, "Graph neural networks for air traffic flow prediction," *IEEE Access*, vol. 8, pp. 134984–134994, 2020.

[13] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting," in *Proceedings of the 27th international joint conference on artificial intelligence (IJCAI)*, AAAI Press, 2018, pp. 3634–3640.

[14] R. W. Vos, J. Sun, and J. M. Hoekstra, "A transformer-based trajectory prediction model to support air traffic demand forecasting," in *Proceedings of the international conference on research in air transportation (ICRAT)*, 2024.

[15] "Air traffic flow representation and prediction using transformer neural networks," in *SESAR innovation days*, 2022.

[16] S. Yoon and K. Lee, "MAIFormer: Multi-agent inverted transformer for flight trajectory prediction," *arXiv preprint arXiv:2509.21004*, 2025.

[17] F. Tao and M. Zhang, "Digital twin shop-floor: A new shop-floor paradigm towards smart manufacturing," *IEEE Access*, vol. 7, pp. 7427–7438, 2019.

[18] A. Fuller, Z. Fan, C. Day, and C. Barlow, "Digital twin: Enabling technologies, challenges and open research," *IEEE Access*, vol. 8, pp. 108952–108971, 2020.

[19] Federal Aviation Administration, "System wide information management (SWIM)." https://www.faa.gov/air_traffic/technology/swim, 2025.

[20] NASA Ames Research Center, "Digital twin simulator of the national airspace system (NAS)." https://technology.nasa.gov/patent/TOP2-325; NASA, 2024.

[21] Airspace Intelligence, "Flyways AI: Air traffic management platform." https://www.airspace-intelligence.com/solutions/air-traffic-management; Airspace Intelligence, 2025.

[22] M. Rodriguez, "Using real-time data and machine learning to improve air traffic management operations." https://mosaicatm.com/2024/12/16/machine-learning-for-air-traffic-management/; Mosaic ATM, Dec. 2024.

[23] Liu *et al.*, "Digital twin-enabled delay diagnosis traceability and propagation process for airport flight ground service," *International Journal of Intelligent Systems*, 2024, doi: 10.1155/int/7458758.

[24] National Oceanic and Atmospheric Administration, "NCEI access services API: Global hourly weather dataset." https://www.ncei.noaa.gov/access/services/data/v1; NOAA National Centers for Environmental Information, 2025.

[25] National Oceanic and Atmospheric Administration, "Integrated surface database (ISD) station history metadata file." https://www.ncei.noaa.gov/pub/data/noaa/isd-history.csv; NOAA National Centers for Environmental Information, 2025.

[26] OurAirports, "OurAirports global airport database." https://davidmegginson.github.io/ourairports-data/airports.csv; OurAirports, 2025.